

同济大学计算机系

人工智能课程设计（四）实验报告



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 胡亮

1. 使用开源项目，部署后，跑通实验流程，实现文生图。并详细说明代码原理、各种操作、算法流程。

1.1 实验目的

本部分实验基于开源的 Stable Diffusion v1.5 模型，完成模型部署与文本生成图像的流程复现。实验输入古诗“孤舟蓑笠翁，独钓寒江雪”，结合水墨画风格提示词，生成具有中国传统水墨意境的图像。该部分实验有助于熟悉 Stable Diffusion 文生图模型的基本使用方法，理解提示词、反向提示词、随机种子、采样步数、引导系数等参数在图像生成过程中的作用。

1.2 实验环境

本实验使用 Python 与 PyTorch 深度学习框架完成。主要依赖包括 torch、diffusers、transformers、accelerate、peft 等库。其中，diffusers 用于加载 Stable Diffusion 管线并完成图像生成，transformers 用于配合 Stable Diffusion 中的文本编码器处理文本提示词，PyTorch 负责底层张量计算与 GPU 加速。

本次运行环境如下：

Python: 3.10.20

PyTorch: 2.11.0+cu128

CUDA Runtime: 12.8

GPU: NVIDIA GeForce RTX 5070

模型: stable-diffusion-v1-5/stable-diffusion-v1-5

1.3 Stable Diffusion 文生图基本原理

Stable Diffusion 是一种基于潜空间扩散模型的文本生成图像模型。与直接在像素空间中生成图像不同，Stable Diffusion 先通过变分自编码器将图像压缩到较低维度的潜空间中，再在潜空间中进行扩散与去噪，最后再将潜变量解码回图像。这样可以显著降低计算量，提高生成效率。

其文生图过程可以概括为以下几个步骤：

第一，文本提示词经过 tokenizer 分词，再输入 CLIP 文本编码器，得到文本语义向量。该语义向量用于描述用户希望生成的内容和风格。

第二，模型在潜空间中初始化一组随机噪声。随机噪声由随机种子控制，因此固定随机种子后，生成结果具有较好的可复现性。

第三，UNet 网络在调度器的控制下进行多次去噪。每一步中，UNet 根据当前噪声潜变量、时间步信息和文本条件，预测噪声成分，并逐步将随机噪声还原为符合提示词语义的图像潜变量。

第四，模型使用 classifier-free guidance 机制增强文本控制能力。该机制同时利用条件预测和无条件预测，引导生成结果更加符合提示词。参数 `guidance_scale` 用于控制文本提示词对生成结果的影响程度。较高的影响程度通常会让图像更贴合提示词，但过高时也可能导致图像不自然或细节失真。

第五，VAE 解码器将最终潜变量还原为 RGB 图像，得到最终生成结果。

1.4 提示词设计

由于输入的古诗本身语义凝练，如果仅直接输入中文诗句，模型可能难以准确理解其中的画面元素。另一方面，Stable Diffusion v1.5 使用的文本编码器 CLIP 的训练语料主要为英文，实际使用中对英文提示词的响应通常更稳定。因此，本实验在中文古诗文本的基础上补充英文描述，将诗句中的关键意象转化为更明确的视觉提示，包括“小舟”“渔翁”“寒江”“雪景”“远山”“留白”“宣纸”“水墨晕染”等元素。这样既保留了古诗原文，也增强了模型对具体画面内容和水墨风格的理解。

代码中构造提示词和反向提示词的函数如下：

```
POEM = "孤舟蓑笠翁，独钓寒江雪"
```

```
def build_prompt(poem: str) -> tuple[str, str]:
    """根据古诗内容构造文生图提示词和反向提示词。"""
    prompt = (
        f"Chinese poem: {poem}. "
        "Chinese ink wash painting, xuan paper, black ink, ink diffusion, "
        "brush strokes, blank space, lone fisherman in a small boat, "
        "snowy river, misty mountains"
    )
    negative_prompt = (
        "low quality, blurry, photorealistic, colorful, cartoon, oil painting, "
        "modern city, car, crowd, text, watermark, signature, frame"
    )
    return prompt, negative_prompt
```

其中，正向提示词用于强化水墨画风格和古诗本身的意象；反向提示词用于抑制低质量、模糊、写实摄影、颜色过多、卡通、油画、现代城市、文字水印等不符合实验目标的风格和内容。

1.5 代码实现流程

首先，程序导入 PyTorch、Stable Diffusion 管线、DPM-Solver 调度器以及公共工具函数：

```
import torch
from diffusers import DPMSolverMultistepScheduler, StableDiffusionPipeline

from utils.course4_common import (
    DEFAULT_SEED,
    MODEL_ID,
    get_runtime_environment,
    make_output_dir,
    set_random_seed,
    write_json,
)
```

在公共工具文件中定义默认模型名称和随机种子：

```
MODEL_ID = "stable-diffusion-v1-5/stable-diffusion-v1-5"
DEFAULT_SEED = 2452769
```

为了增强实验结果的稳定性，程序在生成图像前固定 Python、NumPy 和 PyTorch 的随机种子：

```
def set_random_seed(seed: int, *, allow_tf32: bool = True) -> None:
    """固定随机种子，降低训练和采样结果波动。"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
        torch.backends.cuda.matmul.allow_tf32 = allow_tf32
    torch.backends.cudnn.benchmark = False
```

然后，程序创建输出目录，并根据当前硬件环境选择运行设备。如果检测到 CUDA，则使用 GPU 进行推理，并将数据类型设置为 torch.float16，以减少显存占用并提高推理速度。

```
set_random_seed(SEED)
output_dir = make_output_dir(__file__, "question1")

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32

prompt, negative_prompt = build_prompt(POEM)
```

接着，程序加载 Stable Diffusion v1.5 预训练模型：

```
pipe = StableDiffusionPipeline.from_pretrained(
    MODEL_ID,
    torch_dtype=dtype,
    variant="fp16" if device == "cuda" else None,
    local_files_only=True,
)
```

其中，`torch_dtype=dtype` 用于指定模型参数和计算使用的数据类型；当使用 CUDA 时，`variant="fp16"` 表示加载半精度模型权重。`local_files_only=True` 表示模型从本地缓存中读取，不在运行时重新下载权重。

加载模型后，程序将默认调度器替换为 `DPMSolverMultistepScheduler`：

```
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
pipe.enable_attention_slicing()
pipe.set_progress_bar_config(disable=True)
pipe = pipe.to(device)
```

`DPMSolverMultistepScheduler` 是一种常用于扩散模型采样的多步调度器，可以在较少采样步数下获得较好的生成质量。`enable_attention_slicing()` 用于降低注意力计算过程中的显存占用，使模型在显存相对有限的设备上也能稳定运行。最后，`pipe.to(device)` 将模型移动到对应设备上执行。

Stable Diffusion v1.5 使用的 CLIP tokenizer 存在最大 token 长度限制。为了避免提示词被截断，程序在生成图像前检查了正向提示词和反向提示词的 token 数量：

```
prompt_token_count = len(pipe.tokenizer(prompt, truncation=False).input_ids)
negative_token_count = len(pipe.tokenizer(negative_prompt,
truncation=False).input_ids)
max_token_count = pipe.tokenizer.model_max_length
```

```
if prompt_token_count > max_token_count:
    raise ValueError("提示词超过 CLIP 最大长度限制，会导致内容被截断。")
if negative_token_count > max_token_count:
    raise ValueError("反向提示词超过 CLIP 最大长度限制，会导致内容被截断。")
```

本次实验中，正向提示词 token 数量为 72，反向提示词 token 数量为 32，均未超过最大长度 77，因此不会发生提示词截断问题。

最后，程序调用 Stable Diffusion 管线生成图像：

```
generator = torch.Generator(device=device).manual_seed(SEED)

image = pipe(
    prompt=prompt,
    negative_prompt=negative_prompt,
    width=512,
```

```
height=512,  
num_inference_steps=30,  
guidance_scale=7.5,  
generator=generator,  
) .images[0]
```

其中，width 和 height 指定输出图像大小为 512×512 ；num_inference_steps=30 表示采样步数为 30；guidance_scale=7.5 表示文本提示词的引导强度；generator 传入固定随机种子的随机数生成器，从而控制初始噪声。

生成完成后，程序将图片保存到输出路径。

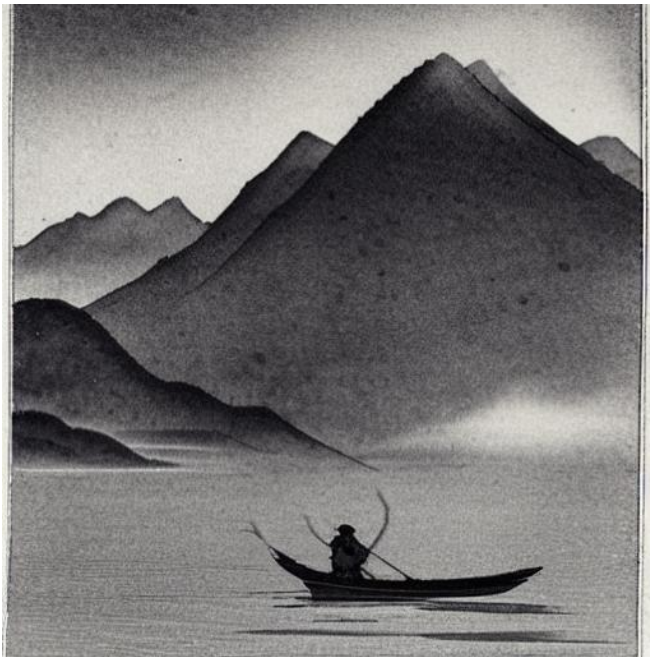
```
image_path = output_dir / "q1_poem_ink_fishing_snow.png"  
image.save(image_path)
```

1.6 实验结果

本次输入诗句为：

孤舟蓑笠翁，独钓寒江雪

生成结果为：



从生成结果来看，画面整体呈现黑白的水墨画风格，没有出现彩色、现代建筑、文字水印等明显干扰元素。图像上半部分由层叠的山体构成，下半部分是空旷的江面，横向水纹和大面积的灰白留白营造出了寒江的清冷感。画面底部出现一叶小舟，船上的人物以深色剪影的形式呈现，整体构图对应了“孤舟”“独钓”“寒江”等核心意象，较好地还原了诗句所表达的孤寂、清冷的氛围。

不过，生成结果仍存在一定不足。首先，“雪”的表现并不明确，画面更多体现了雾气、宣纸颗粒感和灰色的留白，而不是清晰的雪花或雪景。其次，人物细节刻画较弱，船上只能勉强辨认出一个渔翁的轮廓，蓑衣、斗笠和钓竿等与诗句直接相关的细节不够清晰。整体而言，预训练 Stable Diffusion v1.5 能够在提示词的引导下生成符合水墨画风格和古诗意境的图像，但对古诗细节意象的精确刻画仍有提升空间。

2. 在(1)的基础上，通过学习开源项目，写类似的代码（可以参考源代码、并非直接运行已有代码），进行加载数据集（jpg-csv 格式或 parquet 格式数据集）和模型的微调。

2.1 实验目的

本部分在第一小问复现 Stable Diffusion v1.5 文生图流程的基础上，进一步完成图文配对数据集加载与模型微调。实验参考 Stable Diffusion 与 diffusers 中常见的训练流程，但没有直接运行开源项目中已有的训练脚本，而是重新组织了数据整理、CSV 标注读取、图像预处理、文本 token 化、模型组件加载、LoRA 注入、扩散模型训练循环和微调后图像生成等代码。

本小问使用中国山水画图文数据集，将原始图片和文本描述整理为 jpg-csv 格式，并对 Stable Diffusion v1.5 的 U-Net 部分进行 LoRA 微调，使模型进一步适应水墨山水画风格。

2.2 数据集来源与版权说明

本实验使用的数据集为 Hugging Face 平台上的 AmazarashiEndure/Chinese_Landscape_Painting，链接如下：

https://huggingface.co/datasets/AmazarashiEndure/Chinese_Landscape_Painting

该数据集包含中国山水画图像及其对应文本描述，适合用于水墨风格图文生成任务。数据集页面标注许可证为 cc-by-4.0，即 Creative Commons Attribution 4.0 International。该许可证允许在署名的前提下复制、再分发和改编材料。因此，本实验在报告中注明数据来源与许可证信息，并仅用于课程设计中的模型微调与效果展示，符合其开源许可要求，不存在版权使用问题。

2.3 jpg-csv 数据集构建

原始数据集中，每张图片对应一个同名文本描述文件。为了符合题目中“jpg-csv 格式数据集”的要求，程序首先遍历原始数据目录，收集 PNG 图片及其对应 TXT 文本描

述，然后将图片统一转换为 JPEG 格式，并生成 metadata.csv 文件。CSV 文件包含两列：file_name 表示图像相对路径，text 表示对应文本描述。

收集图文配对数据的代码如下：

```
def collect_raw_pairs(raw_root: Path) -> list[tuple[Path, Path]]:
    """收集原始 PNG 图片及同名 TXT 文本描述。"""
    image_paths = sorted(raw_root.rglob("*.png"))
    if not image_paths:
        raise FileNotFoundError(f"未找到 PNG 图像: {raw_root}")

    pairs: list[tuple[Path, Path]] = []
    for image_path in image_paths:
        caption_path = image_path.with_suffix(".txt")
        if caption_path.is_file():
            pairs.append((image_path, caption_path))
        else:
            raise FileNotFoundError(f"缺失文本描述: {caption_path}")
    return pairs
```

整理 jpg-csv 数据集的核心代码如下：

```
def prepare_jpg_csv_dataset(config: TrainingConfig) -> Path:
    """将 PNG/TXT 原始数据整理为 jpg-csv 数据集。"""
    raw_root = config.dataset_root / "dataset"
    prepared_dir = config.prepared_dataset_dir
    image_dir = prepared_dir / "images"
    metadata_path = prepared_dir / "metadata.csv"

    pairs = collect_raw_pairs(raw_root)
    image_dir.mkdir(parents=True, exist_ok=True)
    records: list[dict[str, str]] = []

    for image_path, caption_path in tqdm(pairs, desc="整理 jpg-csv 数据集"):
        jpg_name = make_jpg_name(image_path, raw_root)
        jpg_path = image_dir / jpg_name

        with Image.open(image_path) as image:
            image = ImageOps.exif_transpose(image).convert("RGB")
            image.save(jpg_path, format="JPEG", quality=config.jpeg_quality,
optimize=True)

        records.append(
            {
                "file_name": str(Path("images") / jpg_name).replace("\\", "/"),
                "text": read_caption(caption_path),
```

```

    }
)

prepared_dir.mkdir(parents=True, exist_ok=True)
with metadata_path.open("w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["file_name", "text"])
    writer.writeheader()
    writer.writerows(records)

return metadata_path

```

通过上述处理，原始图文数据被转换为标准的 jpg-csv 形式，后续训练只需要读取 metadata.csv 即可获得图片路径和文本描述。

2.4 数据集加载与预处理

读取 CSV 文件后，程序使用 CsvImageTextDataset 类加载图像和文本。每个样本包含两部分：一是经过预处理的图像张量 pixel_values，二是经过 tokenizer 编码后的文本 token input_ids。

```

class CsvImageTextDataset(Dataset[dict[str, torch.Tensor]]):
    """从 metadata.csv 加载图像路径和文本描述。"""

    def __getitem__(self, index: int) -> dict[str, torch.Tensor]:
        row = self.metadata.iloc[index]
        image_path = self.dataset_dir / str(row[self.file_column])
        caption = str(row[self.text_column])

        pixel_values = load_transformed_image(image_path, self.image_transform)
        input_ids = tokenize_caption(self.tokenizer, caption)

        return {
            "pixel_values": pixel_values,
            "input_ids": input_ids,
        }

```

图像预处理包括缩放、中心裁剪、转换为张量和归一化。由于 Stable Diffusion v1.5 的常用训练分辨率为 512×512 ，本实验也将训练图像统一处理到该尺寸。

```

def build_image_transform(resolution: int) -> transforms.Compose:
    """构造 Stable Diffusion 训练图像预处理流程。"""
    return transforms.Compose(
        [
            transforms.Resize(resolution, interpolation=InterpolationMode.BICUBIC,
                               antialias=True),
            transforms.CenterCrop(resolution),

```

```

        transforms.ToTensor(),
        transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]),
    ]
)

```

文本描述通过 CLIP tokenizer 转换为固定长度 token 序列，作为扩散模型训练时的文本条件：

```

def tokenize_caption(tokenizer: CLIPTokenizer, caption: str) -> torch.Tensor:
    """将文本描述转换为 CLIP token。"""
    tokenized = tokenizer(
        caption,
        max_length=tokenizer.model_max_length,
        padding="max_length",
        truncation=True,
        return_tensors="pt",
    )
    return tokenized.input_ids[0]

```

2.5 模型加载与微调设置

训练时，程序加载 Stable Diffusion v1.5 的调度器、tokenizer、文本编码器、VAE 和 U-Net。VAE 负责将图像编码到潜空间，文本编码器负责将文本描述转换为语义向量，U-Net 负责在扩散过程中预测噪声。

本实验采用 LoRA 方式对 U-Net 进行微调。训练过程中，VAE、文本编码器和 U-Net 原有参数均被冻结，只在 U-Net 注意力层中加入可训练的 LoRA 参数。这样可以在不全量更新原模型的情况下完成风格适配。

```

text_encoder.requires_grad_(False)
vae.requires_grad_(False)
unet.requires_grad_(False)

lora_config = LoraConfig(
    r=config.lora_rank,
    lora_alpha=config.lora_alpha,
    init_lora_weights="gaussian",
    target_modules=["to_k", "to_q", "to_v", "to_out.0"],
    lora_dropout=config.lora_dropout,
)
unet.add_adapter(lora_config)

```

本次训练的主要参数如下：

训练分辨率：512

训练样本数上限：512

batch size: 1

梯度累积步数: 4
最大训练步数: 200
学习率: 1.0e-4
LoRA rank: 8
LoRA alpha: 16
LoRA dropout: 0.05
优化器: AdamW
学习率调度器: CosineAnnealingLR

2.6 训练过程

扩散模型训练的核心目标是让 U-Net 学会预测加入到潜变量中的噪声。训练时，程序首先将图像输入 VAE，得到其在潜空间的表示；然后随机采样噪声和时间步，将噪声加入潜变量；接着将文本 token 输入文本编码器，得到文本条件向量；最后由 U-Net 根据带有噪声的潜变量、时间步和文本条件预测噪声，并与真实噪声计算均方误差损失。

核心训练代码如下：

```
with torch.no_grad():
    latents = vae.encode(pixel_values).latent_dist.sample()
    latents = latents * vae.config.scaling_factor

    noise = torch.randn_like(latents)
    timesteps = torch.randint(
        0,
        noise_scheduler.config.num_train_timesteps,
        (latents.shape[0],),
        device=device,
        dtype=torch.long,
    )

    noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)
    encoder_hidden_states = text_encoder(input_ids, return_dict=False)[0]

    model_pred = unet(
        noisy_latents,
        timesteps,
        encoder_hidden_states,
        return_dict=False,
    )[0]

    loss = F.mse_loss(model_pred.float(), noise.float(), reduction="mean")
```

由于单张 512×512 图像训练时显存占用较高，本实验设置 batch size 为 1，并使用梯度累积。每累计 4 次反向传播后执行一次参数更新：

```

(loss / config.gradient_accumulation_steps).backward()
accumulation_step += 1

if accumulation_step % config.gradient_accumulation_steps == 0:
    torch.nn.utils.clip_grad_norm_(lora_parameters, max_norm=1.0)
    optimizer.step()
    lr_scheduler.step()
    optimizer.zero_grad(set_to_none=True)
    global_step += 1

```

训练过程中，程序将每一步的损失值和学习率记录到 `q2_train_log.csv`。该文件包含 `step`、`loss` 和 `learning_rate` 三列，共记录 200 个训练 `step`。由于扩散模型训练时每一步会随机采样噪声和时间步，损失值存在一定波动，但整体保持在较低范围，最后一步损失约为 0.1241，说明训练过程正常完成。

2.7 微调后图像生成

完成微调后，程序重新加载 Stable Diffusion v1.5 推理管线，并加载第二小问得到的 LoRA 权重：

```

pipe = StableDiffusionPipeline.from_pretrained(
    config.model_id,
    torch_dtype=torch.float16,
    variant="fp16",
    local_files_only=True,
    safety_checker=None,
    requires_safety_checker=False,
)
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
pipe.load_lora_weights(lora_path.parent, weight_name=lora_path.name)

```

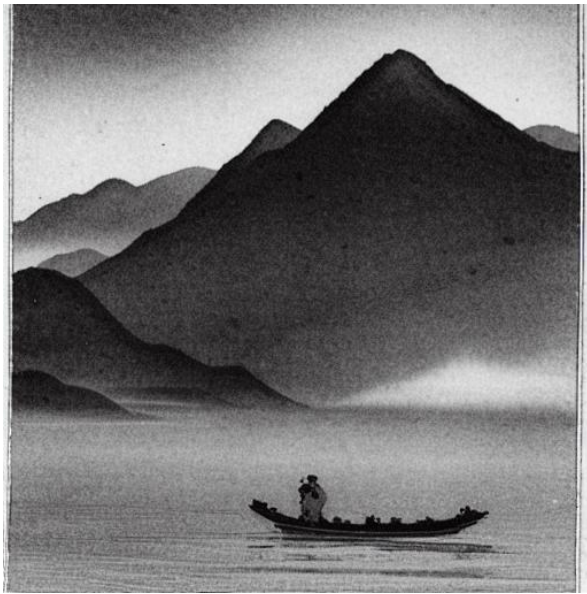
随后使用与第一小问相近的古诗水墨画提示词生成图像：

```

validation_prompt = (
    "Chinese poem: 孤舟蓑笠翁，独钓寒江雪。 ",
    "Chinese ink wash landscape painting, xuan paper, black ink, ",
    "misty mountains, snowy river, lone fisherman in a small boat, ",
    "blank space, elegant brush strokes"
)
negative_prompt = (
    "low quality, blurry, photorealistic, colorful, cartoon, oil painting, ",
    "modern city, car, crowd, text, watermark, signature, frame"
)

```

2.8 实验结果与分析



从生成结果来看，微调后的模型仍然较好地捕捉到了“孤舟蓑笠翁，独钓寒江雪”的整体意境。但不足之处与第一问相似：首先，画面中“雪”的表现仍不够清晰；渔翁的细节不明显，蓑衣、斗笠、钓竿等与诗句直接相关的元素没有被充分刻画出来。整体而言，本次 LoRA 微调使模型保持了较稳定的水墨山水风格，但对诗句中细节意象的精确理解能力仍然有限。

3. 自行寻找其他数据集，进行模型的微调，比如生成写实画

3.1 实验目的

本部分实验在前两小问的基础上，自行寻找新的图文数据集进行模型微调。与第二小问使用中国山水画数据集不同，本小问选择了写实照片风格的数据集，目标是让 Stable Diffusion v1.5 生成更接近真实摄影风格的图像，并观察不同数据集对模型生成风格的影响。

本实验使用的数据集为 Hugging Face 平台上的 KoalaAI/StockImages-CC0，链接如下：

<https://huggingface.co/datasets/KoalaAI/StockImages-CC0>

该数据集包含 stock image 图像及对应文本标签，适合用于写实照片风格的图文生成任务。数据集许可证为 CC0，允许个人和商业使用，且不要求署名。因此，本实验仅将该数据集用于实验中的模型微调训练，不存在版权问题。

3.2 parquet 数据集加载

KoalaAI/StockImages-CC0 数据集以 parquet 格式存储。本实验使用其中的 image 字段作为图像输入，使用 tags 字段作为文本描述。

程序首先读取本地 parquet 文件：

```
def get_parquet_files(dataset_root: Path) -> list[str]:
    """获取本地 parquet 数据文件。"""
    parquet_files = sorted((dataset_root / "data").glob("*.parquet"))
    if not parquet_files:
        raise FileNotFoundError(f"未找到 parquet 数据文件: {dataset_root / 'data'}")
    return [str(path) for path in parquet_files]
```

随后使用 datasets 库加载 parquet 数据，并过滤无效样本。为了控制训练开销，本实验最多选取 512 个样本参与训练：

```
def load_realistic_dataset(config: TrainingConfig) -> HFDataset:
    """加载写实风格 parquet 图文数据集。"""
    data_files = get_parquet_files(config.dataset_root)
    dataset = load_dataset("parquet", data_files=data_files, split="train")

    required_columns = {config.image_column, config.text_column}
    if not required_columns.issubset(dataset.column_names):
        raise ValueError("parquet 数据集必须包含图像字段和文本字段。")

    dataset = dataset.cast_column(config.image_column, HFImage(decode=False))
    dataset = dataset.filter(
        lambda row: sample_is_valid(row, config),
        desc="过滤无效训练样本",
    )

    if config.max_train_samples is not None and len(dataset) >
config.max_train_samples:
        dataset =
dataset.shuffle(seed=config.seed).select(range(config.max_train_samples))

    return dataset
```

由于 parquet 中的图像可能以字节形式保存，程序实现了专门的图像读取函数，将图像统一转换为 RGB 格式：

```
def load_parquet_image(image_value: Any) -> Image.Image:
    """从 parquet 图像字段读取 RGB 图像。"""
    if isinstance(image_value, Image.Image):
        return ImageOps.exif_transpose(image_value).convert("RGB")
```

```

if isinstance(image_value, Mapping):
    image_bytes = image_value.get("bytes")
    if image_bytes:
        with Image.open(io.BytesIO(image_bytes)) as image:
            return ImageOps.exif_transpose(image).convert("RGB")

```

```

raise TypeError("parquet 图像字段无法读取为 PIL Image。")

```

为了强化写实照片风格，程序在原始标签前加入统一文本前缀：

```

text_prefix: str = "realistic photo, stock photography"

```

3.3 微调设置与生成

本小问仍采用 LoRA 方法进行微调，微调对象为 Stable Diffusion v1.5 的 U-Net 注意力层。与第二小问不同的是，本次训练数据从水墨风格的山水画换成了写实风格的照片数据，因此模型学习目标也从水墨风格转向了写实风格。

本次训练主要参数如下：

模型：stable-diffusion-v1-5/stable-diffusion-v1-5

数据集：KoalaAI/StockImages-CC0

数据格式：parquet

图像字段：image

文本字段：tags

训练样本数上限：512

训练分辨率：512

最大训练步数：200

学习率：1.0e-4

LoRA rank：8

LoRA alpha：16

LoRA dropout：0.05

训练过程中，程序将损失值和学习率记录到 q3_train_log.csv。该文件共记录 200 个训练步，最后一步损失约为 0.1251，说明训练过程正常完成。训练结束后，LoRA 权重保存为：

```

course4_outputs/question3/lora_weights/q3_unet_lora.safetensors

```

完成微调后，程序加载训练得到的 LoRA 权重，并使用写实照片风格提示词生成结果图像：

```

validation_prompt = (
    "Chinese poem: 孤舟蓑笠翁，独钓寒江雪。 "
    "realistic photo, cinematic winter landscape, snowy river, "
    "lone fisherman in a small boat, misty mountains, natural light, "
    "high detail, documentary photography"
)

```

```
)  
negative_prompt = (  
    "low quality, blurry, cartoon, anime, illustration, oil painting, "  
    "ink wash painting, watercolor, text, watermark, signature, frame"  
)
```

3.4 实验结果与分析



从生成结果来看，模型已经明显学习到了写实照片风格。画面中出现了蓝天、雪山、积雪树林、冬季河流和自然光照，整体色彩、光影和空间透视都更接近真实摄影。图像能够体现“寒江雪”中的雪山、寒冷江面和冬季环境，但诗句中的核心主体出现了明显缺失：画面中没有孤舟、渔翁、蓑笠和垂钓元素，变成了一张冬季湖泊/山谷风光照片。由此可见，写实照片数据集能够有效改变模型的生成风格，但并没有增强模型对古诗意象的理解能力。

4. 分析模型的实际效果，指出如何改进模型，进一步提升效果。

4.1 实际效果分析

结合前三小问的生成结果来看，Stable Diffusion v1.5 在古诗转图像任务中能够部分捕捉到诗句的整体意境，但实际效果仍然不够理想。第一小问的原始模型能够生成黑白水墨风格的画面，画面中有远山、江面、小舟和人物剪影，基本符合“寒江”“孤舟”的意象，但雪景、蓑衣、斗笠、钓竿等细节不清晰。第二小问经过中国山水画数据集 LoRA 微调后，水墨画山水风格呈现效果仍然稳定，构图也与第一小问相似，但画面细节没有明显增强。第三小问使用写实图片数据集微调后，模型成功生成了写实照片风格的雪山、树

林、河流和自然光照，说明数据集确实改变了模型风格；但画面中缺少孤舟、渔翁、蓑笠和垂钓元素，更像一张冬季山谷风光照片，而不是“独钓寒江雪”的可视化图片。整体来看，模型对图像风格的学习效果较成功，但对古诗中细节意象的理解和控制能力不足。

造成上述问题的原因主要有以下几点：

1. 训练数据和目标语义之间不匹配。中国山水画数据集更适合学习水墨风格，写实图库更适合学习照片质感、光照和自然场景，但它们都不一定包含“雪中小船、老人、蓑衣斗笠、独自垂钓”等组合场景。LoRA 微调主要强化数据集中高频出现的视觉模式，因此模型更容易生成雪山、湖泊和树林，而不是在数据集里低频出现或缺失的诗句主体“孤舟蓑笠翁”。
2. 第三小问使用的 tags 多为关键词列表，缺少完整句式的图文描述，模型更容易学习“snow, mountain, lake, winter”这类松散标签与图像的对应关系，而不是“一个渔翁在雪中小舟上垂钓”这样的对象关系。
3. Stable Diffusion v1.5 对中文诗句的理解能力有限，虽然提示词中保留了古诗原文，但实际生成更依赖后续英文提示词；如果英文提示中主体不够突出，模型会优先生成更常见的雪景风光。
4. “远处小船上的渔翁”本身属于小目标，在 512×512 分辨率下容易被忽略，蓑衣、斗笠和鱼竿等细节更难稳定生成。

4.2 改进方向

1. 改进数据集，使训练数据更贴近目标场景。可以补充雪景河流、小船、渔夫、钓鱼、冬季山水等图像，尤其是“人物在船上垂钓”的样本，而不仅仅依赖通用山水画或通用写实风图库。对于水墨风格任务，可以寻找更高质量的中国画图文数据；对于写实风格任务，可以加入冬季垂钓、小舟、穿蓑笠的老人等更具体的照片样本。这样模型在微调时才能真正学习到目标主体，而不只是学习水墨或写实风格。
2. 可以改进文本标注质量。相比关键词标签，完整自然语言 caption 更适合训练文生图模型。例如可以将简单标签改写为：“a lone elderly fisherman wearing a bamboo hat and straw rain cape, fishing from a small wooden boat on a snowy river, with misty mountains in the background”。这种描述明确包含人物、服饰、动作、船、环境和背景关系，比单独的 snow, mountain, lake, winter 更有利于模型学习复杂的场景组合。
3. 可以优化提示词。当前提示词虽然包含古诗原文和风格描述，但主体强调仍然不足。后续生成时可以将关键主体放在提示词最前面，并适当重复强调，例如：“a lone fisherman in a small wooden boat, wearing a bamboo hat and straw raincoat, fishing on a snowy river, Chinese ink wash painting, misty mountains, cold winter atmosphere。”这样可以提高模型对“渔翁、小舟、垂钓”这些核心元素的注意力。

4. 可以采用多随机种子生成并筛选结果。单张图像受到随机初始噪声影响较大，不能完全代表模型上限。对于同一组提示词和 LoRA 权重，可以使用多个 seed 生成多张候选图，再选择主体最完整、风格最符合要求的作为最终结果。
5. 可以在数据质量可靠的前提下增加训练样本数、训练步数或 LoRA rank。目前第二、三小问的训练更偏向轻量的风格适配，能够改变画面风格，但不足以稳定学习复杂的主体。增加训练强度可能改善模型对特定元素的表达，但也需要注意过拟合和风格退化的问题。

5. 调整模型参数，生成更好效果的图片。或换用其他生成模型，尝试效果。

5.1 实验目的

前几小问均以 Stable Diffusion v1.5 为基础模型进行文生图和 LoRA 微调，但生成结果在人物、小船、钓竿等细节上存在质量问题。本部分选择后者：换用其他生成模型的方式进行尝试，使用 SDXL Base 1.0 重新生成古诗水墨画，并通过多随机种子生成候选图，比较不同 seed 对生成结果的影响。

5.2 模型与参数设置

本实验使用 SDXL Base 1.0 作为基础模型。相比 Stable Diffusion v1.5，SDXL 在图像细节、构图和提示词响应方面通常能力更强，因此适合用于进一步提升生成效果。

本次生成参数如下：

模型：stabilityai/stable-diffusion-xl-base-1.0

生成尺寸：1216 × 832

采样步数：36

guidance_scale: 7.8

随机种子：2452769, 2452770, 2452771, 2452772

采样器：DPMSolverMultistepScheduler

提示词如下：

```
prompt = (  
    "Poem: 孤舟蓑笠翁，独钓寒江雪。"  
    "Chinese ink wash painting, shanshui landscape, lone elderly fisherman, "  
    "bamboo hat, straw rain cape, small boat on snowy river, "  
    "misty mountains, xuan paper, black ink, empty space"  
)  
  
negative_prompt = (
```

```

    "photorealistic, western oil painting, colorful, modern city, crowd, "
    "motorboat, anime, cartoon, blurry, low quality, bad hands, text, watermark"
)

```

其中，正向提示词将“孤舟”“渔翁”“斗笠”“蓑衣”“雪江”“远山”“宣纸”“留白”等元素显式写出，避免模型只依赖中文古诗进行理解；反向提示词则排除写实摄影、油画、彩色、现代城市、卡通、水印等不符合水墨画任务的内容。

5.3 SDXL 推理实现

程序使用 StableDiffusionXLPipeline 加载 SDXL Base 1.0，并将调度器设置为 DPM-Solver 多步采样器：

```

pipe = StableDiffusionXLPipeline.from_pretrained(
    config.model_id,
    torch_dtype=torch.float16,
    variant="fp16",
    use_safetensors=True,
    local_files_only=config.local_files_only,
)

pipe.scheduler = DPMSolverMultistepScheduler.from_config(
    pipe.scheduler.config,
    use_karras_sigmas=True,
    algorithm_type="sde-dpmsolver++",
)

pipe.vae.enable_slicing()
pipe.enable_model_cpu_offload()

```

其中，torch.float16 用于降低显存占用，vae.enable_slicing() 和 enable_model_cpu_offload() 用于进一步缓解 SDXL 推理时的显存压力。随后，程序对多个随机种子分别进行生成：

```

for seed in config.seeds:
    generator = torch.Generator(device="cuda").manual_seed(seed)
    image = pipe(
        prompt=config.prompt,
        negative_prompt=config.negative_prompt,
        width=config.width,
        height=config.height,
        num_inference_steps=config.num_inference_steps,
        guidance_scale=config.guidance_scale,
        generator=generator,
    ).images[0]

```

```
image.save(sample_dir / f"q5_sd-xl_ink_seed_{seed}.png")
```

多 seed 生成可以在同一提示词和模型条件下得到不同构图结果，便于从候选图中选择主体更完整、画面更符合诗意的一张。

5.4 实验结果与分析



四个随机种子的生成结果来看，换用 SDXL Base 1.0 后，图像质量和意象表达相比前几小问有了明显提升。四张图都呈现出较稳定的黑白水墨风格，包含山脉、江面、雾气、小舟和渔翁等元素，整体已经能够较好呈现“孤舟蓑笠翁，独钓寒江雪”所表达的意境。相比之下，seed=2452770 和 seed=2452771 中人物和小舟在画面中更加突出，尤其是斗笠、长杆和船身的轮廓较为明显，但占据比例偏大，传统水墨山水画的空灵感略弱。而 seed=2452769 和 seed=2452772 的画面中，渔翁占据的画面比例明显更少，留白更多，更符合原句的空灵意境。

总体而言，SDXL 对“水墨山水”“留白”“孤舟”“渔翁”等核心意象的表现更稳定，比 Stable Diffusion v1.5 更容易生成完整的船、渔翁和钓竿等核心元素。不过，生成结果仍存在一些局限：部分图像出现了不可读的伪书法和伪印章，这是模型在模拟中国画版式时常见的视觉伪影；同时，“雪”的表现仍然主要依赖灰白留白和寒冷氛围，而不是明确的落雪或积雪细节。综合比较后，seed=2452769 和 seed=2452772 的构图更自然，主体与环境比例更协调，更适合作为本小问的代表结果。

6. 对 stable diffusion 的其他组件进行微调（比如文本编码器），或对 stable diffusion 进行 Dreambooth Lora 微调，以减少其对显存的占用。并展示微调前后效果与差异。

6.1 实验目的

本部分选择对 Stable Diffusion v1.5 进行 DreamBooth LoRA 微调。与全量微调不同，LoRA 微调不更新原模型的全部参数，而是在模型的部分注意力层中加入少量可训练的低秩适配器参数。本实验冻结 VAE、文本编码器和 U-Net 原始参数，仅训练 U-Net 中的 LoRA 参数，从而减少训练过程中的显存和存储开销。

本实验的目标是：在保持 Stable Diffusion v1.5 基础生成能力的同时，让模型结果更偏向古诗的水墨画场景，并比较其在 DreamBooth LoRA 微调前后的生成效果差异。

6.2 DreamBooth 训练样本选择

本实验沿用第二小问整理得到的中国山水画 jpg-csv 数据集，并从中筛选少量更适合 DreamBooth LoRA 微调的样本。为了使训练样本尽量贴近“孤舟蓑笠翁，独钓寒江雪”的主题，程序根据文本描述中的关键词对样本打分，优先选择包含 fishing、fisherman、boat、river、water、misty 等词语的图文样本。

关键词权重设置如下：

```
keyword_weights: tuple[tuple[str, int], ...] = (  
    ("fishing", 6),  
    ("fisherman", 6),  
    ("boat", 5),  
    ("river", 3),  
    ("water", 2),  
    ("misty", 1),  
)
```

样本打分代码如下：

```
def score_metadata(metadata: pd.DataFrame, config: TrainingConfig) -> pd.Series:  
    """根据目标意象关键词为样本打分。"""  
    text = metadata[config.text_column].astype(str).str.lower()  
    score = pd.Series(0, index=metadata.index, dtype="int64")  
    for keyword, weight in config.keyword_weights:  
        score += text.str.contains(keyword, regex=False).astype("int64") * weight  
    return score
```

最终程序选取 24 个样本用于 DreamBooth LoRA 微调。与第二小问直接使用较多图文样本训练不同，本小问更强调少量与主题相关的样本对特定风格和意象的绑定。

6.3 DreamBooth LoRA 微调设置

DreamBooth 训练时，程序没有直接使用原始 caption，而是为所有训练样本统一设置实例提示词，使模型将这些样本共同绑定到一个特定风格标识 qfhshuimo style 上：

```
instance_prompt: str = (  
    "qfhshuimo style, Chinese ink wash landscape painting, misty mountains, "  
    "river, small boat, xuan paper, black ink, empty space"  
)
```

验证生成时使用的提示词为：

```
validation_prompt: str = (  
    "Poem: 孤舟蓑笠翁，独钓寒江雪. qfhshuimo style, "  
    "Chinese ink wash painting, lone fisherman, bamboo hat, small boat, "  
    "snowy river, misty mountains, black ink, empty space"  
)
```

其中，qfhshuimo style 用于调用 DreamBooth LoRA 学到的水墨山水风格；后面的 lone fisherman、bamboo hat、small boat、snowy river 等词语用于强调古诗中的关键主体元素。

本次 DreamBooth LoRA 微调主要参数如下：

基础模型：stable-diffusion-v1-5/stable-diffusion-v1-5

训练样本数：24

训练分辨率：512

batch size: 1

梯度累积步数：4

最大训练步数：240

学习率：1.0e-4

LoRA rank: 8

LoRA alpha: 16

LoRA dropout: 0.05

是否训练文本编码器：否

6.4 显存开销降低的实现方式

本实验通过冻结原模型参数、只训练 LoRA 适配器来减少显存占用。训练模块加载后，程序首先冻结文本编码器、VAE 和 U-Net 的原始参数：

```
text_encoder.requires_grad_(False)  
vae.requires_grad_(False)
```

```
unet.requires_grad_(False)
```

随后仅在 U-Net 注意力层中的 to_k、to_q、to_v 和 to_out.0 等模块加入 LoRA 适配器：

```
lora_config = LoraConfig(
    r=config.lora_rank,
    lora_alpha=config.lora_alpha,
    init_lora_weights="gaussian",
    target_modules=["to_k", "to_q", "to_v", "to_out.0"],
    lora_dropout=config.lora_dropout,
)
unet.add_adapter(lora_config)
```

训练时，优化器只接收仍然需要梯度更新的 LoRA 参数：

```
lora_parameters = [parameter for parameter in unet.parameters() if
    parameter.requires_grad]

optimizer = torch.optim.AdamW(
    lora_parameters,
    lr=config.learning_rate,
    betas=(config.adam_beta1, config.adam_beta2),
    weight_decay=config.adam_weight_decay,
    eps=config.adam_epsilon,
)
```

这样做的好处是，训练过程中不需要为完整的 U-Net、VAE 和文本编码器保存全部可训练参数的梯度与优化器状态，只需更新少量新增 LoRA 参数。实验程序在 DreamBooth LoRA 训练开始前重置 CUDA 峰值显存统计，并在训练结束后读取峰值显存，本次训练阶段的 CUDA 峰值显存约为 3198.23 MB。训练完成后保存的 LoRA 权重文件约为 6.1 MB，也明显小于完整 Stable Diffusion 模型权重文件。由此可见，本实验采用的 DreamBooth LoRA 微调方式能够以较低显存和较小存储开销完成模型适配。

6.5 训练与生成流程

DreamBooth LoRA 的训练过程与前面 LoRA 微调的基本扩散训练流程一致：图像先经过 VAE 编码到潜空间，再加入随机噪声；文本提示词经过文本编码器得到条件向量；U-Net 根据带噪潜变量、时间步和文本条件预测噪声，并通过均方误差损失进行训练。

本实验首先使用未加载 LoRA 的 Stable Diffusion v1.5 生成微调前图片：

```
before_path = generate_sd15_sample(
    config=config,
    prompt=config.validation_prompt,
    negative_prompt=config.negative_prompt,
    image_path=config.output_dir / "samples" / "q6_before_dreambooth.png",
```

```

width=config.validation_width,
height=config.validation_height,
num_inference_steps=config.validation_steps,
guidance_scale=config.validation_guidance_scale,
)

```

随后进行 DreamBooth LoRA 微调，并在训练后加载 LoRA 权重生成微调后图片：

```

lora_path, train_logs = train_lora(
    config=config,
    dataloader_builder=lambda tokenizer: build_dreambooth_dataloader(
        metadata=selected_metadata,
        dataset_dir=config.prepared_dataset_dir,
        tokenizer=tokenizer,
        config=config,
        instance_prompt=config.instance_prompt,
    ),
    train_sample_count=len(selected_metadata),
)

after_path = generate_lora_sample(
    config=config,
    lora_path=lora_path,
    prompt=config.validation_prompt,
    negative_prompt=config.negative_prompt,
    image_path=config.output_dir / "samples" / "q6_after_dreambooth_lora.png",
    width=config.validation_width,
    height=config.validation_height,
    num_inference_steps=config.validation_steps,
    guidance_scale=config.validation_guidance_scale,
)

```

训练过程中，程序记录了 240 个训练 step，最后一步 loss 约为 0.0544，平均 loss 约为 0.1504。由于扩散模型训练时每一步随机采样噪声和时间步，损失值会存在波动，但整体训练过程能够正常完成。

6.6 微调前后效果对比



从对比图可以看出，微调前后的结果都表现出了黑白水墨画风格，画面主体均由远山、江面和小舟构成，整体已经具备“寒江”“孤舟”的基本意境。微调前图像中，小舟上出现了两个人物剪影，其中右侧人物手持长杆，左侧人物的存在也较明显，因此画面更像“双人泛舟”或“船上同行”，与“孤舟蓑笠翁”单人的孤寂感完全不一致；同时人物轮廓较模糊，斗笠、蓑衣和垂钓动作不够明确。微调后图像中，船上主体变为更清楚的单人形象，人物头部有类似斗笠的轮廓，整体更接近“独钓”的单人场景。画面中的长杆从人物附近向远处延伸，比微调前更容易联想到钓竿。

不过，微调后的改善主要体现在主体数量和人物轮廓上，整体画面风格和背景结构变化不大。两张图都以远山和江面为主，雪景仍然没有被清晰地表现出来，更多依靠灰白留白和颗粒质感营造寒冷氛围，而缺少明确的积雪或落雪效果。此外，微调后人物虽然更接近单人渔翁，但蓑衣、钓竿和垂钓动作仍不够精细。总体来看，DreamBooth LoRA 微调在较低显存开销下增强了结果中“小舟上的单人渔翁”这一核心意象，使结果比微调前更贴近诗句主题。